

AI Agents and Agentic AI with Python & Generative AI

Module 1
Agentic AI ConceptsModule 2
AI Agents, Tools, Actions, & LanguageModule 3
GAME: A Conceptual Framework for AI AgentsOverview of the GAME Framework
Video • 5 minDesigning AI Agents with GAME
Ungraded Plugin • 15 minSimulating Agents in ChatGPT
Video • 7 minSimulating GAME Agents in Conversation
Ungraded Plugin • 30 minModular AI Agent Design
Ungraded Plugin • 15 minAgent Loop Customization
Ungraded Plugin • 15 minImplementing GAME in Code
Ungraded Plugin • 15 minTry Out the Agent Framework
Reading • 10 minHow Your Agent Communicates with the LLM:
The Agent Language
Ungraded Plugin • 15 minPutting It All Together: Document Your Code
with a README Agent
Ungraded Plugin • 15 minModule 4
Agent Tool MangementModule 5
Rethinking How Software is Built in the Age of AI Agents

Designing AI Agents with GAME

The GAME Framework: Designing AI Agents

The starting point of an agent should be thinking through its design. While much of our focus has been on implementing code, taking a step back to structure an agent's architecture before writing a single line is crucial. The GAME framework provides a methodology for systematically defining an agent's goals, actions, memory, and environment, allowing us to approach the design in a logical and modular fashion. By thinking through how these components interact within the agent loop, we can sketch out the agent's behavior and dependencies before diving into code implementation. This structured approach not only improves clarity but also makes the transition from design to coding significantly smoother and more efficient.

The **GAME** framework provides a structured way to design AI agents, ensuring modularity and adaptability. It breaks agent design into four essential components:

- **G - Goals / Instructions:** What the agent is trying to accomplish and its instructions on how to try to achieve its goals.
- **A - Actions:** The tools the agent can use to achieve its goals.
- **M - Memory:** How the agent retains information across interactions, which determines what information it will have available in each iteration of the agent loop.
- **E - Environment:** The agent's interface to the external world where it executes actions and gets feedback on the results of those actions.

Goals and instructions are grouped together under "G" because they work in tandem to shape the agent's behavior. Goals specify *what* the agent is trying to achieve, serving as the high-level objectives that define the desired outcomes of the agent's operation. Instructions, on the other hand, provide the *how*, detailing the specific steps, strategies, and constraints that guide the agent toward fulfilling its goals effectively. Together, they form the foundation that ensures the agent not only understands its purpose but also follows a structured approach to accomplishing its tasks.

One important discussion is the relationship between Actions and the Environment. Actions define **what** the agent can do—they are abstract descriptions of potential choices available to the agent. The Environment, on the other hand, determines **how** those actions are carried out, providing concrete implementations that execute within the real-world context of the agent. This distinction allows us to separate high-level decision-making from the execution details, making the agent more modular and adaptable.

You can think of Actions as an "interface" specifying the available capabilities, while the Environment acts as the "implementation" that brings those capabilities to life. For example, an agent might have an action called `read_file()`, which is simply a placeholder in the Actions layer. The Environment then provides the actual logic, handling file I/O operations and error handling to ensure the action is executed correctly. This separation ensures flexibility—agents can be designed to operate across different environments by simply swapping out implementations while keeping their decision logic intact.

Motivating Example: The Proactive Coder

To illustrate how the GAME framework applies in practice, consider an AI agent designed to proactively enhance a codebase. This **Proactive Coder** agent will scan a repository, analyze patterns in the code, and propose potential new features that it could implement with a small number of changes. If the user approves a feature, the agent will generate the initial implementation and suggest refinements.

Using the GAME framework, we break down the agent design:

- **Goals:**
 - Goals (What to achieve):
 - Identify potential enhancements
 - Make sure that the enhancements are helpful and relevant
 - Make sure that the enhancements are small and self-contained so that they can be implemented by the agent with minimal risk
 - Ensure that the changes do not break existing interfaces
 - Ensure that the agent only implements features that the user agrees to
 - Instructions (How to achieve it):
 - Pick a random file in the code base and read through it
 - Read some related files to the original file
 - Read at most 5 files
 - Propose three feature ideas that are implementable in 2-3 functions and require minimal editing of the existing code
 - Ask the user to select which feature to implement
 - List the files that will need to be edited and provide a list of proposed changes for each
 - Go file by file implementing the changes until they are all edited
- **Actions:**
 - List project files
 - Read project file
 - Ask user to select a feature
 - Edit project file
- **Memory:**
 - We will use a simple conversational memory and store the complete contents of files in the conversation for reference
- **Environment:**
 - We will provide simple implementations of the actions in Python to run locally, but could later change to an implementation that works in GitHub Actions.

Expand

Completed

Go to next item

Like Dislike Report an issue

Go to next item →